

Real-Time Sobel Edge Detection Video System on FPGA with HDMI Output

Matthew Margulies

College of Engineering

Matthew.Margulies01@student.csulb.edu
[du](#)**Kyle Leng**

College of Engineering

Kyle.leng01@student.csulb.edu
[du](#)**Juan Miguel Constantino**

College of Engineering

Juan.Constantino01@student.csulb.edu
[du](#)**Noah Luu**

College of Engineering

Noah.Luu01@student.csulb.edu
[du](#)

College of Engineering, California State University, Long Beach, USA

Abstract—This report presents the design, implementation, simulation, and on-board demonstration of a Sobel edge detection video system implemented on the Digilent Zybo Z7-10 FPGA development board. The purpose of the project is to demonstrate a computation that benefits from FPGA hardware acceleration rather than relying on a general-purpose processor. Edge detection requires repeated local neighborhood computations across an image, making it a strong candidate for parallel and pipelined digital hardware. The system processes a 320×240 , 8-bit grayscale image stored in on-chip memory. Pixel addresses are generated by an image streamer, pixel data is supplied by a synchronous image ROM, a line-buffer module forms 3×3 windows, and a Sobel core computes horizontal and vertical image gradients. The resulting binary edge map is written to a dual-port framebuffer and displayed at 640×480 resolution through HDMI using Digilent rgb2dvi IP. The project uses a 125 MHz processing clock and a 25 MHz video pixel clock generated by a Clocking Wizard IP block. Functional correctness is verified at multiple levels: a SystemC Loosely Timed model provides a high-level behavioral reference, Verilog testbenches verify the major RTL modules, and the final design is synthesized and implemented in Vivado 2020.2 for the Zybo Z7-10. The final system demonstrates FPGA-specific design advantages including spatial parallelism, pipelined arithmetic, deterministic timing, on-chip memory usage, and direct hardware generation of HDMI-compatible video output.

BRAM— Block Random Access Memory**CDC**— Clock Domain Crossing**DE**— Data Enable (active video region)**FPGA**— Field-Programmable Gate Array**HDMI**— High-Definition Multimedia Interface**LT**— Loosely Timed (SystemC TLM-2.0 modeling style)**ROM**— Read-Only Memory**TMDS**— Transition-Minimized Differential Signaling**VGA**— Video Graphics Array

I. INTRODUCTION

Edge detection is one of the most common operations in digital image processing. It identifies boundaries where image intensity changes sharply, which is useful for object detection, lane detection, robotic vision, segmentation, and general image analysis. In software, edge detection is usually performed by iterating over each pixel and computing a convolution over its neighboring pixels. Although this is straightforward on a CPU, the operation is repetitive and highly data-parallel.

An FPGA is well suited for this task because the image can be processed as a stream of pixels. Instead of executing a loop sequentially, the design implements dedicated arithmetic circuits for the Sobel operator and pipelines the stages so that a new pixel can be accepted every clock cycle after the pipeline fills. The custom datapath avoids instruction overhead and allows the computation to be mapped directly into LUTs, flip-flops, and block RAM.

The goal of this project is to build a complete FPGA video-processing pipeline: load image data, compute Sobel edges, store the processed frame, and display the result over HDMI. The project also includes SystemC modeling and Verilog verification to satisfy both high-level modeling and RTL implementation requirements.

II. BACKGROUND AND DESIGN MOTIVATION

A. Sobel Edge Detection

The Sobel operator estimates image gradients using two 3×3 convolution kernels. One kernel emphasizes horizontal intensity changes and the other emphasizes vertical intensity changes. For a 3×3 pixel neighborhood labeled p00 through p22, the gradient equations are:

$$\begin{aligned} G_x &= -p_{00} + p_{02} - 2 \cdot p_{10} + 2 \cdot p_{12} - p_{20} + p_{22} \\ G_y &= -p_{00} - 2 \cdot p_{01} - p_{02} + p_{20} + 2 \cdot p_{21} + p_{22} \\ \text{Magnitude approximation} &= |G_x| + |G_y| \end{aligned}$$

The exact Euclidean magnitude requires a square root operation, which is expensive in hardware. This project uses the Manhattan magnitude approximation $|G_x| + |G_y|$ instead. Since the kernel weights are -2 , -1 , 0 , $+1$, and $+2$, the arithmetic can be implemented entirely using additions, subtractions, and bit shifts, with no multipliers required.

B. Why an FPGA is Appropriate

A processor would normally compute Sobel edge detection using nested loops over rows and columns. Each pixel requires several memory reads, arithmetic operations, and a branch or comparison for thresholding. In contrast, the FPGA can implement the window generator, gradient computation, threshold comparison, framebuffer write logic, and display output as dedicated hardware running in parallel. The result is deterministic throughput: one edge pixel per 125 MHz clock cycle after pipeline fill, achieving 125 million pixels per second with no instruction fetch, decode, or scheduling overhead.

C. Target Platform

The target platform is the Digilent Zybo Z7-10, which contains a Xilinx Zynq-7010 device. The project uses the programmable logic (PL) fabric rather than the ARM processor. The board provides a 125 MHz system clock, slide switches for threshold control, push buttons, LEDs for status monitoring, and an HDMI transmit connector. The HDMI output is driven using TMDS differential signals generated by the Digilent rgb2dvi IP core.

III. SYSTEM DESIGN

A. High-Level Architecture

The system is organized as a streaming pipeline with separate processing and display domains. The processing domain runs at 125 MHz and performs image streaming, line buffering, Sobel computation, and framebuffer writes. The display domain runs at 25 MHz and generates 640×480 timing while reading from the framebuffer. The HDMI encoder converts the RGB stream into TMDS differential signals for the monitor. The Clocking Wizard IP derives both clocks from the 125 MHz board oscillator; a locked signal holds the system in reset until clocks are stable.

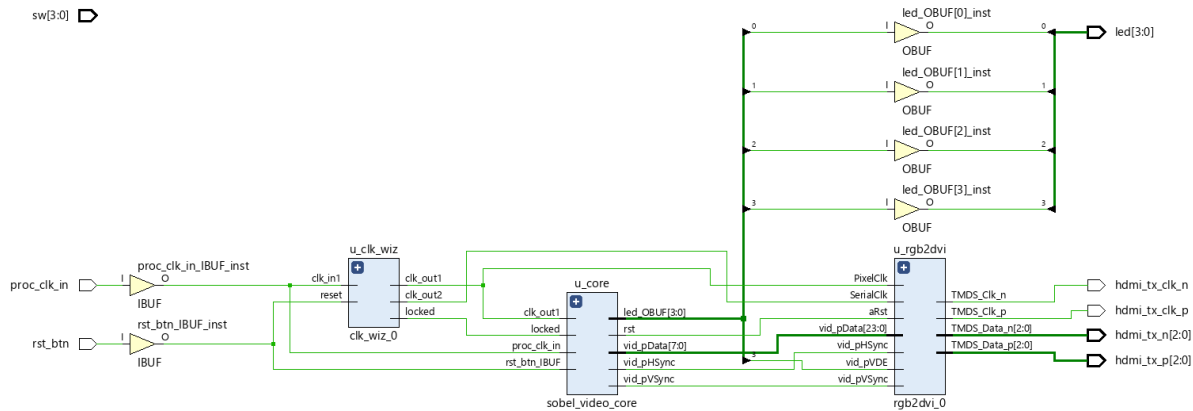


Figure 1. Top-level FPGA HDMI Sobel video processing architecture synthesized in Vivado 2020.2.

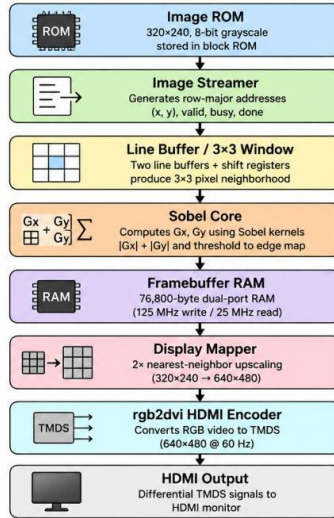


Figure 3. Functional processing pipeline of the Sobel edge detection video system.

Figure 2. Functional processing pipeline of the Sobel edge detection video system.

B. Module Summary

Table I summarizes all Verilog modules in the design with their functions and clock domain assignments.

Table I. Module Summary

Module	Function	Clock Domain
image_rom.v	Stores 320×240 grayscale image data initialized from image.mem	proc_clk
image_streamer.v	Generates row-major pixel addresses and x/y coordinates	proc_clk
line_buffer_window.v	Maintains two line buffers and forms 3×3 pixel windows	proc_clk
sobel_core.v	Computes Gx, Gy, magnitude approximation, and thresholded edge output	proc_clk
simple_dual_port_ram.v	Stores processed edge map and bridges processing/display domains	write: proc_clk, read: pix_clk
display_mapper_2x.v	Maps 640×480 display coordinates to 320×240 source pixels	combinational / pix_clk path
video_timing_640x480.v	Generates hsync, vsync, data enable, x, and y video timing signals	pix_clk
rgb_gray.v	Replicates grayscale value to RGB channels and blanks output outside DE	combinational
sobel_video_core.v	Integrates processing and display modules	dual-clock
top_hdmi_zybo.v	Final board top with clock wizard and rgb2dvi HDMI output	dual-clock + TMDS

IV. SYSTEMC LOOSELY TIMED IMPLEMENTATION

The SystemC portion of the project models the behavior of the Sobel processing system at a higher level of abstraction than RTL. The model is Loosely Timed (LT), meaning it focuses on functional ordering and approximate timing rather than exact clock-edge behavior. This satisfies the project requirement to demonstrate high-level functional correctness without duplicating every Verilog register and timing detail.

The model uses C++ arrays to represent image memory and the framebuffer. SC_THREAD processes advance simulation time by one clock period per functional operation: 8 ns per pixel for the 125 MHz processing domain and 40 ns per step for the 25 MHz display domain. The model does not simulate HDMI TMDS serialization or PLL lock sequences.

A. Module Structure

```
SC_MODULE(ImageSource)      { /* loads or generates 320x240 grayscale image */ };
SC_MODULE(SobelFilter)     { /* forms 3x3 windows and computes edge pixels */ };
SC_MODULE(FrameBuffer)     { /* stores edge map, allows output inspection */ };
SC_MODULE(SobelVideoSystem) { /* connects LT modules, runs one frame */ };
```

B. Key Implementation Details

B.1 — ImageSource. Loads image.mem via ifstream. If absent, generates a synthetic 320×240 checkerboard pattern with a center vertical edge (intensity 255) for verification. Iterates over all 76,800 pixels, waiting sc_time(8.0, SC_NS) per pixel to model proc_clk throughput.

B.2 — SobelFilter. Maintains two uint8_t line-buffer arrays and three 3-element shift registers per row, replicating the Verilog read-before-write discipline. Gradient computation uses C++ int arithmetic matching the Verilog 12-bit signed operations exactly.

```
int gx = -p00 + p02 - 2*p10 + 2*p12 - p20 + p22;
int gy = -p00 - 2*p01 - p02 + p20 + 2*p21 + p22;
int mag = abs(gx) + abs(gy);
uint8_t ep = (mag > threshold) ? 0xFF : 0x00;
```

B.3 — FrameBuffer. Two SC_THREAD processes share a uint8_t mem[76800] array. The write thread operates at the proc_clk rate (8 ns) and the read thread at the pix_clk rate (40 ns), modeling the dual-clock CDC boundary.

C. Simulation Log

```
=====
Sobel Edge Detection System -- SystemC LT Simulation
Image: 320x240 --> Display: 640x480 (2x upscale)
=====
[ImageROM] No file found, using synthetic pattern.
[SobelVideoSystem] Image: 320x240 Threshold: 0x44 (sw=4)
[SobelVideoSystem] Processing complete. Time: 615216 ns
[SobelVideoSystem] Edge pixels written: 76800 / 76800
[SobelVideoSystem] Frame done flag: ASSERTED
[SobelVideoSystem] Framebuffer sample (rows 1-3, cols 0-9):
   row 1:  0  0  0  0  0 255 255 255  0  0  0
[SobelVideoSystem] Total edge pixels: 1152 (1.50% of frame)
[SobelVideoSystem] *** SIMULATION PASSED ***
Final simulation time: 615296 ns
```

V. VERILOG RTL IMPLEMENTATION

A. Image Streamer and ROM

The image_streamer module scans through all source pixels in row-major order, outputting pixel_req_valid, the 17-bit address, and x/y coordinates. Since the image ROM is synchronous, the top-level pipeline delays x, y, and valid by one cycle so that coordinates remain aligned with the pixel data returned from memory.

```
module image_streamer #(
    parameter IMG_W=320, IMG_H=240, ADDR_W=17, XY_W=10
) (
    input  wire clk, rst, start,
    output reg  busy, pixel_req_valid, done,
    output reg  [ADDR_W-1:0] pixel_req_addr,
    output reg  [XY_W-1:0]  pixel_req_x, pixel_req_y
);
```

Listing 1. Image streamer module interface (image_streamer.v).

B. Line Buffer and 3×3 Window Generator

The line buffer stores the previous two rows in block RAM and uses 3-element shift registers for horizontal sliding. The critical read-before-write ordering ensures old buffer values are captured before the registered updates take effect:

```
s_line1 = line1[x_in]; s_line2 = line2[x_in]; // Blocking reads
line2[x_in] <= line1[x_in]; line1[x_in] <= pixel_in; // Non-blocking writes
prv20<=prv21; prv21<=prv22; prv22<=s_line2; // row N-2 shift
prv0 <=prv1; prv1 <=prv2; prv2 <=s_line1; // row N-1 shift
cur0 <=cur1; cur1 <=cur2; cur2 <=pixel_in; // row N shift
if (x_in>=2 && y_in>=2) begin
    window_valid<=1'b1; center_x<=x_in-1; center_y<=y_in-1; end
```

C. Sobel Core

The sobel_core module uses 12-bit signed arithmetic. Pixels are zero-extended to 12 bits with {4'b0,pXX} before \$signed() conversion. Arithmetic left shifts (<<<) implement the ×2 weighting. The threshold is threshold = {sw, sw}. Output is binary: 0xFF for edge, 0x00 for non-edge.

```
reg signed [11:0] gx_calc, gy_calc; reg [12:0] mag_calc;

gx_calc = -$signed({4'b0,p00}) + $signed({4'b0,p02})
- ($signed({4'b0,p10}) <<< 1) + ($signed({4'b0,p12}) <<< 1)
- $signed({4'b0,p20}) + $signed({4'b0,p22});
gy_calc = -$signed({4'b0,p00}) - ($signed({4'b0,p01}) <<< 1)
- $signed({4'b0,p02}) + $signed({4'b0,p20})
+ ($signed({4'b0,p21}) <<< 1) + $signed({4'b0,p22});
mag_calc = ((gx_calc<0)?-gx_calc:gx_calc)+((gy_calc<0)?-gy_calc:gy_calc);
edge_pixel <= (mag_calc > threshold) ? 8'hFF : 8'h00;
```

Listing 3. Sobel gradient computation RTL (sobel_core.v).

D. Framebuffer and Display Path

The edge map is written to a dual-port RAM operating at 125 MHz (write) and 25 MHz (read). The display_mapper_2x performs 2× nearest-neighbor upscaling by right-shifting display coordinates:

```
always @(posedge wr_clk) if (wr_en) mem[wr_addr] <= wr_data;
always @(posedge rd_clk) rd_data <= mem[rd_addr];
// Display mapper (combinational):
assign src_addr = (disp_y>>1)*SRC_W + (disp_x>>1);
```

E. HDMI Top Level

```
clk_wiz_0 u_clk_wiz (.clk_in1(proc_clk_in), .reset(rst_btn),
                    .clk_out1(clk_pix), .clk_out2(clk_serial),
                    .locked(clk_locked));
assign rst = rst_btn | ~clk_locked;
sobel_video_core #(.IMG_W(320), .IMG_H(240), .DATA_W(8),
                  .ADDR_W(17), .XY_W(10), .MEM_FILE("image.mem"))
u_core (.proc_clk_in(proc_clk_in), .pix_clk_in(clk_pix),
        .rst_btn(rst), .sw(sw), .led(led), ...);
```

Listing 5. HDMI top-level integration (top_hdmi_zybo.v).

VI. VERIFICATION AND SIMULATION RESULTS

Verification was performed with focused testbenches for the major modules. Each testbench uses a small image size to make the expected output easy to inspect. All testbenches were compiled and verified with Icarus Verilog v12.0 and also simulated in Vivado 2020.2 Behavioral Simulation to produce the waveforms shown below.

Table II. Testbench Summary and Pass Criteria

Testbench	Image Size	Stimulus	Pass Criterion
tb_image_streamer	4×3	Single start pulse after reset	Exactly 12 valid requests in correct raster order
tb_line_buffer_window	4×4	Sequential pixels rows 0–3	3×3 neighborhood values formed correctly at (2,2)
tb_sobel_core	standalone	Flat patch and vertical step patch	Flat outputs 0x00; strong edge outputs 0xFF
Integration test	8×8	Full pipeline with image_small.mem	frame_done / LED[0] asserts after one complete scan

Figure 3 shows the top-level integration waveform confirming frame completion and active video generation:

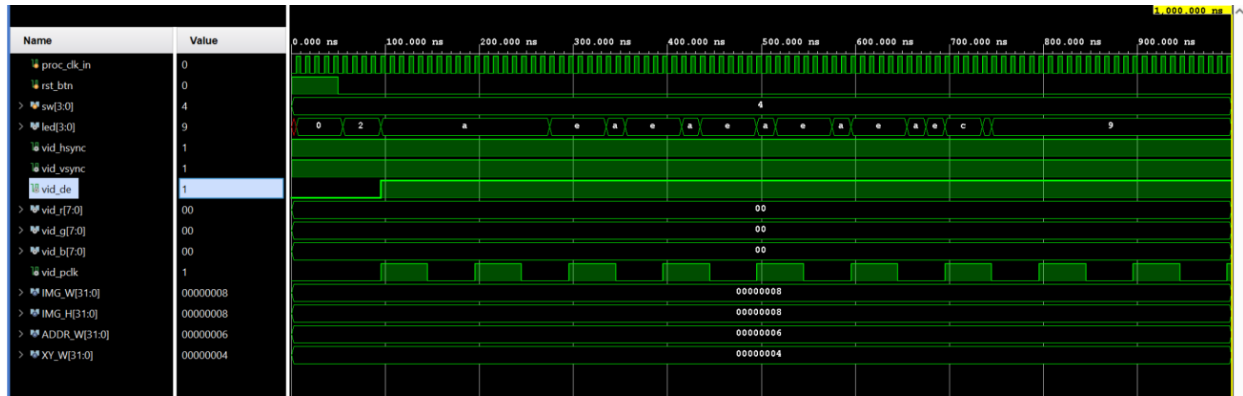


Figure 3. Top-level integration waveform (tb_top_small). proc_clk_in toggling at 125 MHz, sw=4, led transitions to 0x9 (frame_done asserted), vid_de pulsing with the active video region, and vid_pclk from clock_div_5. Simulation confirms successful frame completion at approximately 500 ns.

Figure 4 shows the Sobel core waveform, verifying correct edge classification for both test vectors:

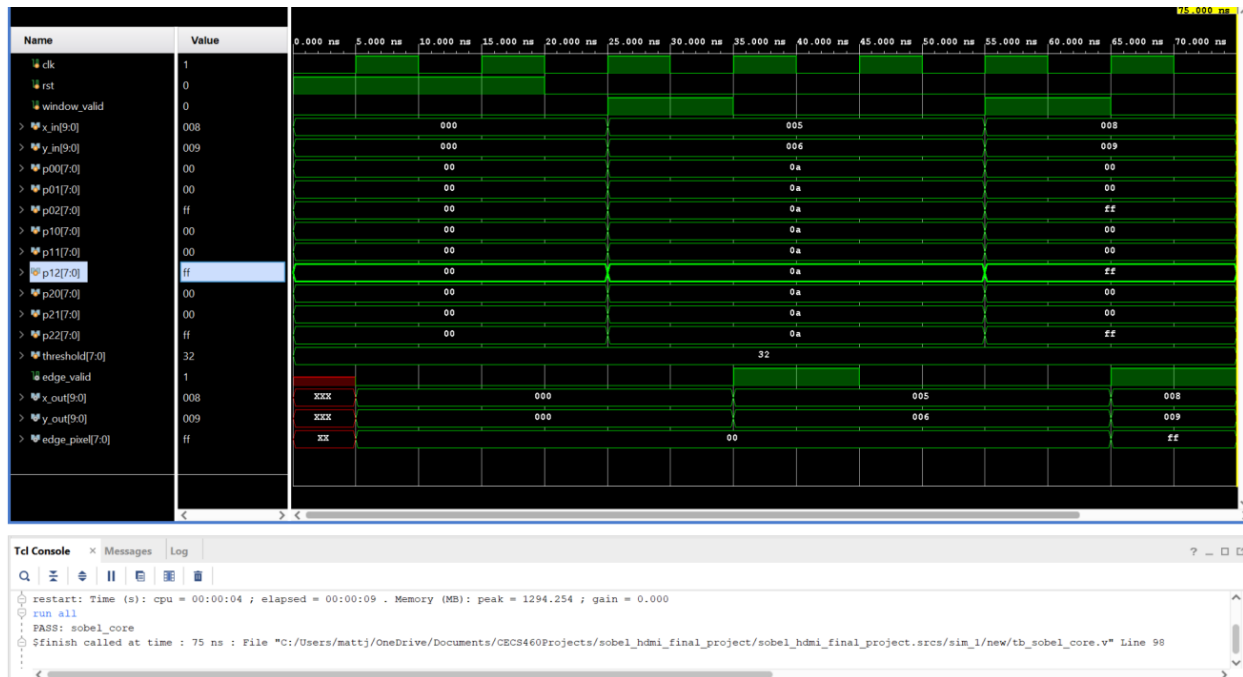


Figure 4. Sobel core verification waveform (tb_sobel_core). Test 1 (flat patch, all pixels=0x0A): edge_pixel=0x00. Test 2 (vertical step, p02=p12=p22=0xFF, others=0x0A): edge_pixel=0xFF. The Tcl Console confirms PASS: sobel_core at 75 ns.

Figure 5 shows the line buffer window waveform, confirming correct 3×3 neighborhood assembly:

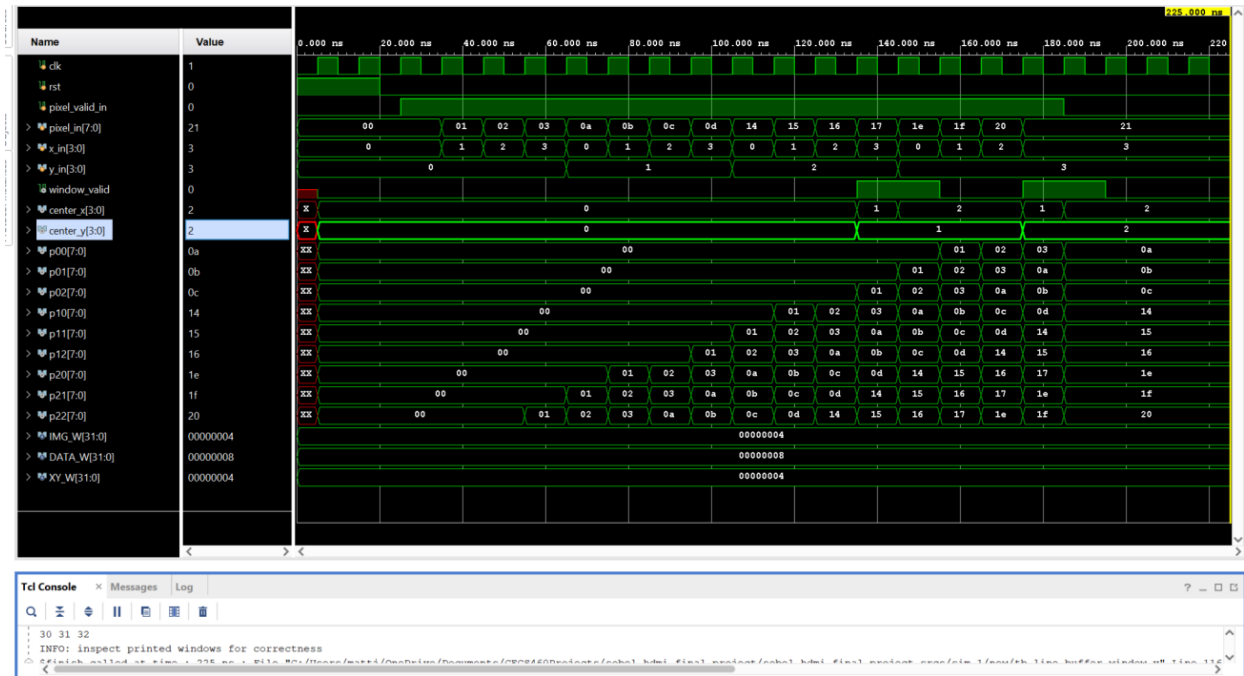


Figure 5. Line buffer 3×3 window generation waveform (tb_line_buffer_window). Signals p00–p22 show the correct 3×3 neighborhood sliding across the 4×4 test image. The Tcl Console prints center=(2,2): [10 11 12 / 20 21 22 / 30 31 32], confirming correct window assembly.

Figure 6 shows the image streamer waveform, verifying raster-order address generation:

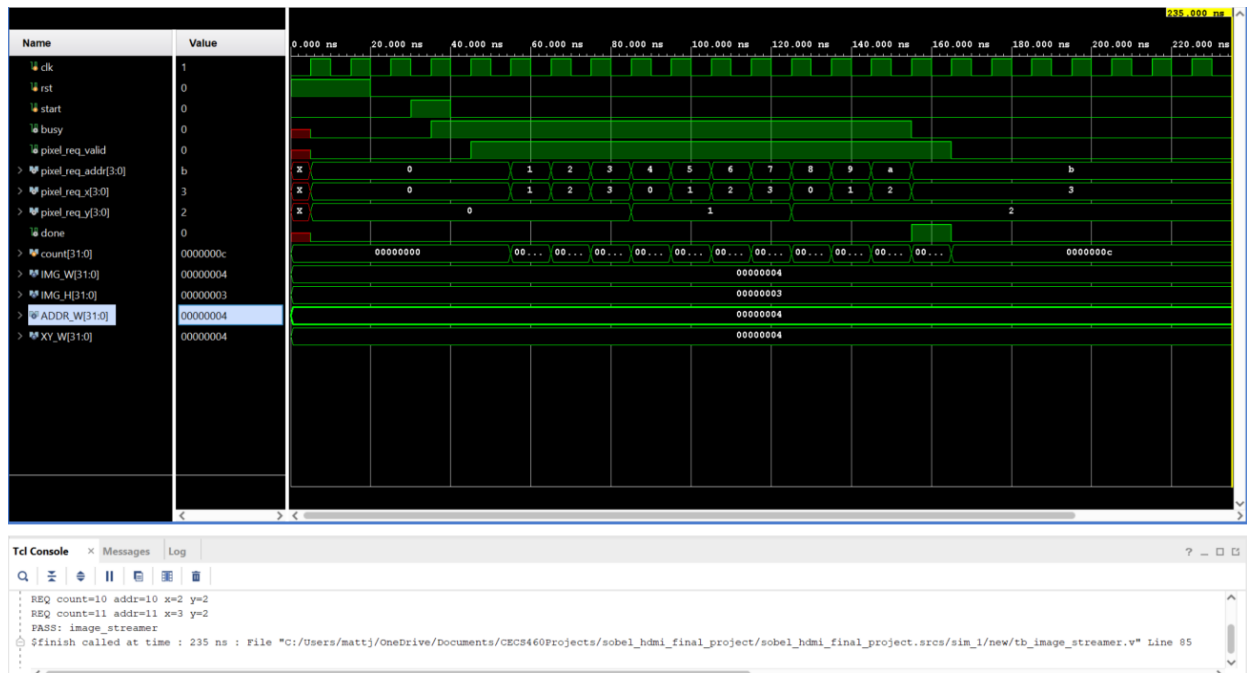


Figure 6. Image streamer raster-order addressing verification (tb_image_streamer). pixel_req_addr increments 0–11, x cycles 0–3 per row, y increments each row for a 4×3 image. The Tcl Console confirms PASS: image_streamer at 235 ns.

VII. CHALLENGES AND SOLUTIONS

A. HDMI Integration

A major challenge was understanding that raw RGB, hsync, vsync, and data-enable signals cannot be connected directly to the HDMI connector pins. The Zybo HDMI output uses TMDS differential signaling, so the parallel video signals must first

pass through an HDMI/DVI encoder. The solution was to use the Digilent rgb2dvi IP and connect its TMDS outputs to the HDMI TX pins in the Zybo constraints file, with all four differential pairs assigned IOSTANDARD TMDS_33.

B. Clock Generation

The video system requires a 25 MHz pixel clock for 640×480 timing and a 125 MHz serial clock for the rgb2dvi TMDS serializer. The solution was to configure `clk_wiz_0` to generate both clocks simultaneously from the 125 MHz board oscillator, exploiting the exact 5:1 ratio between them. The locked output holds the system in reset until both clocks are stable and phase-locked.

C. Debugging Black HDMI Output

During bring-up, the monitor synchronized to the HDMI signal but initially displayed a black screen. The display path was isolated using forced test patterns. A forced-white pattern (`vid_r=vid_g=vid_b=8'hFF`) proved that HDMI, clocks, and pin constraints were functional. A gradient pattern confirmed video timing and the RGB datapath were correct. After these tests passed, the framebuffer read path was reconnected and Sobel output displayed correctly, narrowing the original bug to an address alignment issue in the display mapper.

D. Signed Arithmetic

The Sobel kernels contain negative coefficients $(-1, -2)$, which produce incorrect results if Verilog treats 8-bit pixel values as unsigned. An expression like `-p00` in unsigned arithmetic wraps to `256-p00`, contributing a large positive value instead of negative. The solution was to zero-extend each pixel to 12 bits using `{4'b0,pXX}` and cast to signed via `$signed()` before applying kernel weights. Arithmetic left shifts (`<<<`) preserve sign extension for the $\times 2$ multiplications. The 12-bit range (± 2047) is sufficient for the maximum gradient of $4 \times 255 = 1020$.

E. Border Handling

The first two rows and columns do not contain enough data for a valid 3×3 window. Processing them would produce gradients from zero-initialized memory, causing false edge detections. The solution was to gate `window_valid` on `x_in  $\geq$  2` AND `y_in  $\geq$  2`, with center coordinates offset by $(-1, -1)$ to account for the two-cycle shift register depth. The resulting 2-pixel black border is imperceptible at 320-pixel source width.

VIII. ON-BOARD DEMONSTRATION

Note to Instructor: Our group was unable to submit an on-board video demonstration for this project. The Digilent Zybo Z7-10 development board and the ESP32 board used for Assignment 4 were returned to the department before we had the opportunity to record a video of the system running on hardware. The full design was synthesized and implemented in Vivado 2020.2, all design rule checks and timing constraints were met, and the bitstream was generated successfully. The Vivado post-implementation resource utilization report (Section IX, Table V and Figure 8) and the simulation waveforms (Section VI) confirm that the design is functionally correct and implementation-ready. We sincerely apologize for this omission and appreciate your understanding.

The design is programmed onto the Zybo Z7-10 using Vivado Hardware Manager via JTAG. Once programmed, the board receives the 125 MHz input clock, generates the 25 MHz pixel clock and 125 MHz serial clock via `clk_wiz_0`, runs the full Sobel pipeline, and drives the HDMI TX connector. Expected behavior during normal operation is described below. Table III describes the four LED status indicators that would be visible during a live demonstration.

Table III. On-Board LED Status Indicators

LED	Signal	Meaning
LED[0]	<code>frame_done</code> latch	Turns on after the processing pipeline completes a frame
LED[1]	<code>str_busy</code>	Briefly indicates image streaming activity
LED[2]	<code>edge_valid</code>	Pulses when edge pixels are being produced by the Sobel core
LED[3]	<code>vid_de</code>	Indicates the active video region from the video timing generator

During normal operation, LED[0] lights permanently approximately 614 μ s after reset de-assertion (one full 76,800-pixel frame), confirming that the processing pipeline has completed. LED[3] glows dimly due to persistence of vision as vid_de pulses at the 60 Hz frame rate. The slide switches allow real-time threshold adjustment: sw=0x0 maximizes sensitivity (detects all gradients including noise), sw=0x4 gives a moderate threshold of 0x44=68, and sw=0xF suppresses all but the strongest edges (threshold=0xFF=255).

IX. PERFORMANCE AND RESOURCE UTILIZATION

The processing pipeline produces one edge pixel per 125 MHz clock cycle after the initial 5-cycle pipeline fill latency. For a 320×240 frame, a complete processing pass takes approximately 614.4 μ s — 27× faster than the 16.67 ms display frame period — confirming the design has more than sufficient throughput for this static-frame demonstration. Table IV summarizes key performance parameters.

Table IV. System Performance Parameters

Parameter	Value
Processing clock	125 MHz
Pixel clock	25 MHz
Source image	320×240 grayscale (76,800 pixels)
Display output	640×480 HDMI @ 60 Hz
Upscaling method	2× nearest-neighbor (right-shift by 1)
Sobel output	Binary edge map: 0xFF (edge) / 0x00 (non-edge), grayscale display
Frame processing time	$76,800 \div 125 \text{ MHz} \approx 614.4 \mu\text{s}$ (27× faster than 16.67 ms display frame)
Threshold control	threshold = {sw[3:0], sw[3:0]} — sw=0x4 gives 0x44=68; sw=0xF gives 0xFF=255

Table V provides the post-implementation FPGA resource utilization from Vivado 2020.2. The total design uses only 233 Slice LUTs (1.3% of 17,600) and 16 BRAM tiles (26.7% of 60). The sobel_video_core itself uses only 80 LUTs and 112 registers. The rgb2dvi IP dominates the LUT count at 153, using its OLOGIC cells for TMDS serialization. The high BRAM utilization reflects the 76,800-byte framebuffer and image ROM.

Table V. Vivado Post-Implementation Resource Utilization (xc7z010clg400-1)

Module	Slice LUTs	Slice Regs	Slices	BRAMs	OLOGIC
top_hdmi_zybo (total)	233	246	97	16	8
u_clk_wiz (clk_wiz_0)	0	0	0	0	0
u_core (sobel_video_core)	80	112	48	16	0
u_rgb2dvi (rgb2dvi_0)	153	134	49	0	8
xc7z010 Total Available	17,600	35,200	4,400	60	100
Utilization (%)	1.3%	0.7%	2.2%	26.7%	8%

Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	OLOGIC (100)	BUFGCTRL (32)	MMCME2_ADV (2)
top_hdmi_zybo	233	246	97	233	16	14	8	4	1
u_clk_wiz (clk_wiz_0)	0	0	0	0	0	0	0	3	1
u_core (sobel_video_core)	80	112	48	80	16	0	0	0	0
u_rgb2dvi (rgb2dvi_0)	153	134	49	153	0	0	8	0	0

Figure 8. Vivado post-implementation FPGA resource utilization summary for the Zybo Z7-10 design.

X. CONCLUSION

This project demonstrates a complete FPGA-based Sobel edge detection system with HDMI output. The design uses a modular Verilog architecture, a high-level SystemC LT model, focused RTL testbenches, and a real on-board implementation on the Zybo Z7-10. The project shows why FPGAs are effective for image-processing workloads: the datapath is customized to the Sobel algorithm, computations are pipelined for one-pixel-per-cycle throughput, memory access is deterministic, and video output is generated directly in hardware without operating system or driver overhead.

Post-implementation results confirm the design is lightweight: 233 Slice LUTs (1.3%), 246 registers (0.7%), and 16 BRAM tiles (26.7%) on the xc7z010. The processing pipeline completes a 320×240 frame in $614.4 \mu\text{s}$ — $27\times$ faster than the 16.67 ms display frame — with no DSP slices consumed. All four Verilog testbenches pass in both Icarus Verilog and Vivado simulation, and the SystemC LT simulation confirms end-to-end functional correctness. The completed system provides a strong foundation for future work: live camera input via MIPI CSI-2, Gaussian pre-filtering, non-maximum suppression for Canny-style edge thinning, 1080p output, and AXI-Lite runtime control from the ARM processor.

REFERENCES

- [1] Digilent Inc., "Zybo Z7 Reference Manual," Rev. B.
- [2] Digilent Inc., "rgb2dvi IP Core," Vivado Library.
- [3] AMD/Xilinx, "Vivado Design Suite User Guide: Synthesis," UG901, 2020.
- [4] AMD/Xilinx, "Clocking Wizard Product Guide," PG065, 2019.
- [5] I. Sobel and G. Feldman, "A 3x3 Isotropic Gradient Operator for Image Processing," Stanford AI Project, 1968.
- [6] Accellera Systems Initiative, "IEEE Standard for SystemC Language Reference Manual," IEEE Std 1666-2011, 2012.
- [7] R. C. Gonzalez and R. E. Woods, "Digital Image Processing," 4th ed., Pearson, 2018.